

OBJECT ORIENTED PROGRAMMING (JAVA)

Converted from the original C++ notes — same structure, Java syntax & examples

Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects. It simplifies software development and maintenance by providing the concepts defined below.

Class is a user-defined data type which defines its properties and its functions. A class is only the logical representation of the data. For example, Human being is a class. The body parts of a human being are its properties, and the actions performed by the body parts are known as functions. The class does not occupy any memory space until an object is instantiated.

Java Syntax (for class):

```
class Student {
    public int id;           // data member
    public int mobile;
    public String name;

    public int add(int x, int y) { // member function
        return x + y;
    }
}
```

Object is a run-time entity. It is an instance of the class. An object can represent a person, place, or any other item. An object can operate on both data members and member functions.

Java Syntax (for object):

```
Student s = new Student();
```

Note: In Java, objects are **always** created on the heap using the **new** keyword — there is no equivalent of stack-allocated objects like in C++. The reference variable (e.g. s) is stored on the stack and holds the memory address of the object on the heap. Java has no explicit "delete"; the JVM's Garbage Collector automatically reclaims memory when an object is no longer reachable.

Inheritance

Inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In this way, you can reuse, extend, or modify the attributes and behaviors defined in other classes.

In Java, the class which inherits the members of another class is called the **subclass (derived class)**, and the class whose members are inherited is called the **superclass (base class)**. Inheritance is achieved using the **extends** keyword. The derived class is the specialized class for the base class.

Java Syntax:

```
class DerivedClass extends BaseClass {
    // additional members
}
```

Note: Unlike C++, Java does not have visibility-mode inheritance (private/protected/public base class) — every class is inherited in "public" style by default, and access is instead controlled purely through the member's own access specifier.

Types of Inheritance

1. **Single inheritance:** When one class inherits another class, it is known as single-level inheritance.
2. **Multilevel inheritance:** A process of deriving a class from another derived class.
3. **Hierarchical inheritance:** The process of deriving more than one class from a single base class.
4. **Multiple inheritance:** The process of deriving a new class that inherits attributes from two or more classes.
5. **Hybrid inheritance:** A combination of simple, multiple, and hierarchical inheritance.

Important: Java does **not** support multiple inheritance of classes (a class cannot extend two classes) to avoid the "Diamond Problem". Instead, Java achieves multiple-inheritance-like behavior through **interfaces** — a class can implement multiple interfaces.

Encapsulation

Encapsulation is the process of combining data and functions into a single unit called a class. In encapsulation, the data is not accessed directly; it is accessed through the functions present inside the class. In simpler words, attributes of the class are kept **private**, and public getter and setter methods are provided to manipulate these attributes. Thus, encapsulation makes the concept of data hiding possible (a language feature to restrict access to members of an object, reducing the negative effect of dependencies — achieved in Java using the private, protected, and public access modifiers).

Java Example:

```
class Account {  
    private double balance;    // data hidden from outside  
  
    public double getBalance() {    // getter  
        return balance;  
    }  
  
    public void setBalance(double balance) {    // setter  
        if (balance >= 0) {  
            this.balance = balance;  
        }  
    }  
}
```

Abstraction

We try to obtain an abstract view, model, or structure of a real-life problem, reducing its unnecessary details. By defining the properties of a problem — including the affected data and identified operations — the model abstracted from the problem can become a standard solution for that type of problem. It is an efficient approach since many real-life problems share similar properties.

Data binding: Data binding is a process of binding the application UI and business logic. Any change made in the business logic will reflect directly in the application UI (e.g. in JavaFX or Android's data-binding libraries).

In Java, abstraction is primarily achieved using **abstract classes** and **interfaces**.

Polymorphism

Polymorphism is the ability to present the same interface for differing underlying forms (data types). With polymorphism, each class can have different underlying data. A point shape needs only two coordinates (assuming a two-dimensional space). A circle needs a center and radius. A square or rectangle needs two coordinates for the top-left and bottom-right corners and (possibly) a rotation. An irregular polygon needs a series of lines. Precisely, "Poly" means "many" and "morphism" means "forms".

Types of Polymorphism (IMP)

1. Compile Time Polymorphism (Static)
2. Runtime Polymorphism (Dynamic)

Compile Time Polymorphism

The polymorphism which is implemented at compile time is known as compile-time polymorphism. Example — **Method Overloading**.

Method Overloading: Method overloading is a technique which allows you to have more than one method with the same name but with different functionality. Method overloading is possible on the following basis:

1. The type of the parameters passed to the method.
2. The number of parameters passed to the method.

Note: Unlike C++, in Java the return type of a method **alone** is not enough to overload it — the parameter list (number or type) must differ.

Example:

```
class Add {
    public int add(int a, int b) {
        return (a + b);
    }

    public int add(int a, int b, int c) {
        return (a + b + c);
    }
}

public class Main {
    public static void main(String[] args) {
        Add obj = new Add();
        int res1 = obj.add(2, 3);
        int res2 = obj.add(2, 3, 4);
        System.out.println(res1 + " " + res2);
    }
}
```

Output : 5 9 // add() is an overloaded method with a different number of parameters.

Runtime Polymorphism

Runtime polymorphism is also known as dynamic polymorphism. **Method Overriding** is an example of runtime polymorphism. Method overriding means the child class contains a method that is already present in the parent class. Hence, the child class overrides the method of the parent class. In method overriding, the parent and child classes both contain the same method with a different definition. The call to the method is resolved at runtime, which is why it is known as runtime polymorphism.

Java Sample Code:

```
class BaseClass {
    public void show() {
        System.out.println("Apni Kaksha base");
    }
}

class DerivedClass extends BaseClass {
    // @Override annotation is optional but recommended
    @Override
    public void show() {
        System.out.println("Apni Kaksha derived");
    }
}
```

```

}

public class Main {
    public static void main(String[] args) {
        BaseClass b;
        DerivedClass d = new DerivedClass();
        b = d;
        b.show(); // prints the content of show() declared in derived class
    }
}

```

Output : Apni Kaksha derived

Important: In Java, **every non-static, non-final, non-private method is virtual by default** — there is no separate virtual keyword like in C++. This is why overriding "just works" without any extra keyword on the base class method.

Constructor

A constructor is a special method which is invoked automatically at the time of object creation. It is generally used to initialize the data members of new objects. In Java, a constructor has the **same name as the class** and has **no return type** (not even void).

There can be different types of constructors in Java:

1. **Default constructor:** A constructor which takes no arguments. If you don't define any constructor, Java automatically provides one.
2. **Parameterized constructor:** A constructor which has parameters. It is used to provide different values to distinct objects.
3. **Copy Constructor:** Java has no built-in copy constructor like C++. You must write one yourself, or use `clone()`, to create and initialize an object from another object.

Java Sample Code:

```

class Go {
    int x;

    // parameterized constructor
    Go(int a) {
        x = a;
    }

    // "copy constructor" (user-defined, since Java has none built in)
    Go(Go i) {
        x = i.x;
    }
}

public class Main {
    public static void main(String[] args) {
        Go a1 = new Go(20); // Calling the parameterized constructor
        Go a2 = new Go(a1); // Calling the copy constructor
        System.out.println(a2.x);
    }
}

```

Output : 20

Destructor / finalize()

A destructor works opposite to a constructor; it destructs objects of a class. **Java has no destructors**, because Java manages memory automatically using the Garbage Collector (GC) — there is no manual deallocation like C++'s delete.

Note: Java previously offered a `finalize()` method that the GC could call before reclaiming an object, but it is **deprecated since Java 9** and unreliable (its timing is not guaranteed). The modern recommended approach for cleanup (closing files, sockets, etc.) is the **try-with-resources** statement using the `AutoCloseable` interface.

Example (try-with-resources, the modern equivalent):

```
class Resource implements AutoCloseable {
    Resource() {
        System.out.println("Constructor in use");
    }

    @Override
    public void close() {
        System.out.println("Cleanup in use");
    }
}

public class Main {
    public static void main(String[] args) {
        try (Resource a = new Resource();
            Resource b = new Resource()) {
            // use resources
        }
    }
}
```

Output: Constructor in use Constructor in use Cleanup in use Cleanup in use

'this' Keyword

`this` is a keyword that refers to the current instance of the class. There are several main uses of the `this` keyword in Java:

1. It can be used to refer to the current class's instance variable (especially when a parameter shadows a field).
2. It can be used to pass the current object as an argument to another method or constructor.
3. It can be used to invoke another constructor of the same class (constructor chaining) via `this(...)`.

Java Syntax:

```
class Node {
    int data;
    Node next;

    Node(int x) {
        this.data = x;
        this.next = null;
    }
}
```

Friend Function → Not Available in Java

In C++, a **friend function** can access the private and protected members of a class even though it is not a member of that class. **Java has no friend keyword** and does not support this concept at all, because it would break encapsulation, which Java enforces strictly.

Note: To achieve similar behavior in Java, developers commonly use:

- Placing related classes in the **same package** and using **package-private (default)** access.
- Providing public getter/setter methods.
- Using **nested (inner) classes**, which can access the outer class's private members directly.

Aggregation

Aggregation is a process in which one class holds a reference to another class as an entity/field. It is another way to reuse a class. It is a form of association that represents a **"HAS-A" relationship**.

Java Example:

```
class Address {
    String city;
    Address(String city) { this.city = city; }
}

class Employee {
    String name;
    Address address; // Employee "has-a" Address (aggregation)

    Employee(String name, Address address) {
        this.name = name;
        this.address = address;
    }
}
```

Virtual Function → Method Overriding (default in Java) (IMP)

In C++, a **virtual function** is explicitly marked with the `virtual` keyword so that the correct overridden version is called at runtime through a base pointer. **In Java, this behavior is the default** for all non-static, non-final, non-private methods — there is no separate keyword needed.

1. Every instance method in Java is "virtual" unless marked `final`, `static`, or `private`.
2. When a subclass redefines a method with the same signature as its parent, Java resolves the call at runtime based on the actual object type, not the reference type — this is called **dynamic method dispatch**.
3. The `@Override` annotation is optional but strongly recommended, as it lets the compiler catch mistakes.

Key Points:

- A method marked `static` cannot be overridden (it can only be "hidden" — this is different from overriding).
- A class can have a custom cleanup method, but Java has no destructor and therefore no "virtual destructor" concept.

Java Example:

```
class Base {
    // "virtual" by default in Java (re-defined in the derived class)
    public void print() {
        System.out.println("print base class");
    }
}
```

```

    }

    public final void show() { // final = cannot be overridden
        System.out.println("show base class");
    }
}

class Derived extends Base {
    @Override
    public void print() {
        System.out.println("print derived class");
    }
}

public class Main {
    public static void main(String[] args) {
        Base bptr;
        Derived d = new Derived();
        bptr = d;

        bptr.print(); // dynamic dispatch, resolved at runtime
        bptr.show();  // final method, resolved at compile time
    }
}

```

output : print derived class // (impact of dynamic method dispatch) show base class

Pure Virtual Function → Abstract Method

1. An **abstract method** is not used to perform any task on its own. It only serves as a placeholder.
2. An abstract method is a method declared in the base class that has **no body/definition** in the base class.
3. A class containing at least one abstract method must itself be declared abstract, and cannot be instantiated directly — such classes are known as **abstract classes**.
4. The main objective of the abstract base class is to provide common traits to derived classes and to enable runtime polymorphism through the base reference.

Java Syntax:

```
abstract void display();
```

Java Example:

```

abstract class Base {
    abstract void show();
}

class Derived extends Base {
    @Override
    void show() {
        System.out.println("You can see me !");
    }
}

public class Main {
    public static void main(String[] args) {
        Base bptr;
        Derived d = new Derived();
        bptr = d;
        bptr.show();
    }
}

```

```
}  
}
```

```
// output : You can see me !
```

Abstract Classes

In Java, a class is made abstract by using the `abstract` keyword in the class declaration, and it is made "pure" by declaring at least one of its methods as an abstract method (a method with no body, ending in a semicolon). Its implementation must be provided by the derived (concrete) classes.

Example:

```
// abstract class  
abstract class Shape {  
    abstract void draw();  
}  
  
class Rectangle extends Shape {  
    void draw() {  
        System.out.println("Rectangle");  
    }  
}  
  
class Square extends Shape {  
    void draw() {  
        System.out.println("Square");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Rectangle rec = new Rectangle();  
        Square sq = new Square();  
        rec.draw();  
        sq.draw();  
    }  
}
```

```
Output : Rectangle Square
```

Note: Java also has **interfaces** (via the `interface` keyword), which are similar to fully abstract classes. Since Java 8, interfaces can also have default and static methods with bodies. A class can implement multiple interfaces, which is how Java compensates for not supporting multiple class inheritance.

Packages in Java → (equivalent of C++ Namespaces)

1. A **package** is a logical/physical grouping of related classes, designed to avoid naming conflicts — this plays the same role as a namespace in C++.
2. A package defines the scope in which classes, interfaces, and other types are organized and accessed.
3. The main purpose of using packages in Java is to remove ambiguity and organize code. Ambiguity can occur when two different classes/functions share the same name.
4. For example, if two classes both need an `Add` utility class, they can live in different packages to avoid a naming clash.
5. Java's standard library is organized into packages such as `java.util`, `java.io`, and `java.lang` (imported automatically). The statement `import java.util.*;` brings in classes from the `java.util` package.

Java Example:

```
// user-defined package (would normally live in its own file/folder "add")
package add;

public class AddUtil {
    static int a = 5, b = 5;

    public static int add() {
        return (a + b);
    }
}

// --- in another file ---
import add.AddUtil;

public class Main {
    public static void main(String[] args) {
        int res = AddUtil.add(); // accessing the method inside the package
        System.out.println(res);
    }
}
```

```
// output : 10
```

Access Specifiers (IMP)

Access specifiers (modifiers) are used to define how fields and methods can be accessed from outside the class. Java has **four** levels of access (one more than C++):

1. **Private:** Members declared private can be accessed only within the same class.
2. **Default (package-private):** If no modifier is specified, members can be accessed only within classes in the same package. *(This has no direct C++ equivalent.)*
3. **Protected:** Members can be accessed within the same package, and also by subclasses in other packages (via inheritance).
4. **Public:** Members declared public can be accessed from anywhere.

Key Notes (Java-specific)

- Java has no delete or delete[] — memory is reclaimed automatically by the **Garbage Collector**.
- Java does not support "virtual inheritance" for diamond-shaped hierarchies because it doesn't allow multiple class inheritance in the first place; the same problem is avoided by design when using interfaces with default methods (Java 8+ resolves conflicts by forcing the implementing class to override the ambiguous default method).
- **Method (Function) Overloading:** You can have more than one version of the same method in a class; the versions must differ in their parameter list (number and/or type of parameters).
- **Operator Overloading:** Java does **not** support operator overloading (except the built-in overload of + for String concatenation). This is a notable difference from C++.
- Overloading is resolved at **compile time** (static binding); Overriding is resolved at **run time** (dynamic binding).
- Overloading = same method name, different parameter list, may or may not return the same type, within the same class.
- Overriding = same method name, same parameters, same (or covariant) return type, defined in both a class and its subclass.

OOP **JAVA** INTERVIEW QUESTIONS

Frequently asked in technical interviews — quick revision before the interview

#	Question	Answer
1	What are the four main pillars of OOP?	Encapsulation, Abstraction, Inheritance, and Polymorphism.
2	Why is Java called "partially" object-oriented?	Because Java still uses primitive data types (int, char, double, etc.) that are not objects, unlike a purely object-oriented language where everything is an object.
3	What is the difference between overloading and overriding?	Overloading is same method name with different parameters in the same class, resolved at compile time. Overriding is same method signature in parent and child class, resolved at runtime.
4	Why doesn't Java support multiple inheritance of classes?	To avoid the "Diamond Problem" — ambiguity when a class inherits the same method from two parent classes. Java solves this using interfaces instead.
5	How does Java achieve multiple inheritance then?	Through interfaces. A class can implement multiple interfaces, and since Java 8, interfaces can have default methods, with explicit resolution rules if there's a conflict.
6	What is the difference between an abstract class and an interface?	An abstract class can have both abstract and concrete methods, constructors, and instance variables, and supports single inheritance. An interface (traditionally) only had abstract methods, no constructors, and a class can implement multiple interfaces. Since Java 8/9, interfaces can also have default, static, and private methods, narrowing the gap.
7	Can we instantiate an abstract class?	No. An abstract class cannot be instantiated directly; it must be subclassed, and the subclass must implement all abstract methods (or itself be declared abstract).
8	What is dynamic method dispatch?	The mechanism by which a call to an overridden method is resolved at runtime, based on the actual object type (not the reference type) — this is how Java implements runtime polymorphism.
9	Why doesn't Java have destructors?	Because Java uses automatic Garbage Collection to reclaim memory, so manual deallocation isn't needed. The deprecated <code>finalize()</code> method used to offer similar (unreliable) functionality; <code>try-with-resources</code> and <code>AutoCloseable</code> are the modern replacement for deterministic cleanup.
10	What is encapsulation and how is it implemented in Java?	Encapsulation is bundling data and methods together and restricting direct access to internal state. It's implemented in Java by declaring fields private and exposing controlled access via public getter/setter methods.
11	What is the difference between method overloading and constructor overloading?	Constructor overloading is a specific case of method overloading applied to constructors — having multiple constructors in a class with different parameter lists to initialize objects in different ways.
12	What is the "this" keyword used for?	To refer to the current object instance — commonly to disambiguate instance fields from parameters, to pass the current object to another method, or to call another constructor via <code>this(...)</code> .
13	What is the "super" keyword used for?	

		To refer to the immediate parent class — used to call the parent's constructor (<code>super(...)</code>), access a parent's overridden method, or access a parent's field hidden by the subclass.
14	Can a static method be overridden?	No. Static methods belong to the class, not an instance, so they are hidden (not overridden) if redefined in a subclass — this is resolved at compile time based on reference type, unlike true overriding.
15	What is the difference between "IS-A" and "HAS-A" relationships?	"IS-A" represents inheritance (a Dog IS-A Animal). "HAS-A" represents composition/aggregation (a Car HAS-A Engine).
16	Does Java support operator overloading?	No, except for the built-in overload of the "+" operator, which is used for both numeric addition and String concatenation.
17	What is the difference between composition and inheritance?	Inheritance is an "IS-A" relationship achieved via <code>extends</code> , promoting code reuse through a class hierarchy. Composition is a "HAS-A" relationship where a class contains a reference to another class, offering more flexibility and looser coupling — often summarized as "favor composition over inheritance".
18	What are access modifiers in Java?	<code>private</code> , <code>default</code> (package-private), <code>protected</code> , and <code>public</code> — they control the visibility/accessibility of classes, fields, and methods.
19	Can a constructor be private? What's the use case?	Yes. A private constructor prevents object creation from outside the class and is commonly used in the Singleton design pattern or in utility classes with only static members.
20	What happens if a subclass doesn't implement all abstract methods of its abstract parent?	The subclass itself must also be declared abstract; otherwise, it will result in a compile-time error.